

02 - Nanobot 项目概览

本章目标：全面了解 HKUDS/nanobot 项目的背景、核心特性、设计理念，以及为什么它是面试学习的最佳选择。

目录

- 2.1 项目背景
- 2.2 为什么叫“超轻量级”
- 2.3 核心特性详解
- 2.4 项目数据与里程碑
- 2.5 为什么选 Nanobot 学习
- 2.6 与其他框架的差异化对比
- 2.7 面试话术
- 2.8 本章总结

2.1 项目背景

香港大学数据科学实验室（HKUDS）

HKUDS（The University of Hong Kong Data Science Lab）是香港大学的数据科学研究实验室，在大模型、推荐系统、图神经网络等方向有深厚的学术积累。

Nanobot 是 HKUDS 在 2026 年 2 月开源的项目，定位为一个超轻量级 **AI Agent** 框架。从发布到现在短短两个月内就获得了 37K+ Stars，增长速度惊人。

项目诞生的背景

Nanobot 诞生的时代背景	
问题 1: Agent 框架过于复杂	
└─ LangChain 50万行代码，学习曲线陡峭	
└─ AutoGPT 10万行代码，概念验证但不实用	
└─ 初学者和个人开发者难以入门	
问题 2: 缺少适合亚洲市场的 Agent 框架	
└─ 大多数框架不支持微信/飞书/钉钉	
└─ 国内 LLM（通义千问/DeepSeek）接入不便	
└─ 中文生态支持不足	
问题 3: MCP 协议刚刚成熟，需要原生支持	
└─ 2024年底 Anthropic 发布 MCP 协议	
└─ 2025年各大公司跟进支持	
└─ 需要一个从设计之初就原生支持 MCP 的框架	
Nanobot 的解答:	
→ 4000 行代码，极简但完整	
→ 原生支持微信/飞书/钉钉等 8+ 平台	
→ MCP 原生支持，从第一天就内置	
→ MIT 开源，对所有人免费	

项目定位

Nanobot 的定位非常清晰:

一个任何人都能在 5 分钟内启动、一个周末就能读完全部源码的 **AI Agent** 框架。

它不追求功能全面（那是 LangChain 的定位），也不追求自主通用（那是 AutoGPT 的定位），而是追求极简、实用、可理解。

2.2 为什么叫“超轻量级”

代码量对比

这是 Nanobot 最令人印象深刻的特点——仅用约 4000 行 Python 代码实现了完整的 Agent 框架功能。

v0.1-v0.2.1 更新：

核心代码 ~6,000 行，完整项目 64,496 行（173 个 Python 文件）。

框架	核心代码量	语言	依赖数量	安装大小
Nanobot	~4,000 行	Python	少量	~5MB
LangChain	~500,000 行	Python/JS	大量	~100MB+
AutoGPT	~100,000 行	Python	大量	~50MB+
CrewAI	~30,000 行	Python	中等	~30MB
OpenClaw	~430,000 行	Python	大量	~200MB+
MetaGPT	~50,000 行	Python	大量	~80MB+

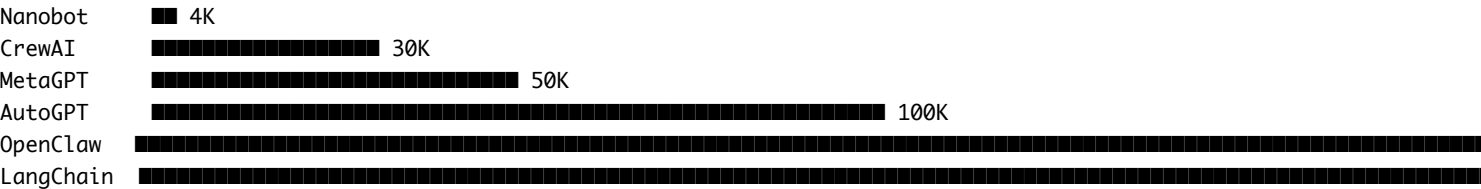
v0.1-v0.2.1 更新：

完整项目代码量对比：

框架	核心代码	完整项目
Nanobot	~6K 行	64K 行
LangChain	~500K 行	500K+ 行
OpenClaw	~430K 行	430K+ 行

可视化对比：

代码行数（K = 千行）



4000 行代码如何做到的？

Nanobot 的极简不是因为功能少，而是因为设计哲学：

1. 利用 **LLM** 的能力，而不是重新造轮子

很多框架花大量代码去实现复杂的规划器（Planner）、任务图（Task Graph）、状态机等。Nanobot 认为：LLM 本身就是最好的规划器，不需要额外的编排引擎。

```
# 其他框架的做法：复杂的任务编排
task_graph = TaskGraph()
task_graph.add_node("search", SearchTool())
task_graph.add_node("analyze", AnalyzeTool())
task_graph.add_edge("search", "analyze")
task_graph.compile().run()

# Nanobot 的做法：让 LLM 自己决定
# LLM 根据上下文自主决定调用什么工具、什么顺序
response = provider.chat_with_retry(messages, tools=available_tools)
```

2. 配置驱动而不是代码驱动

Nanobot 通过 YAML 配置文件来组装功能，而不是用代码硬编码：

```
# nanobot.yml - 一个配置文件定义完整 Agent
name: " 我的助手"
provider:
  model: "gpt-4"
  api_key: "${OPENAI_API_KEY}"
channels:
  - type: telegram
    token: "${TELEGRAM_TOKEN}"
mcp_servers:
  - name: filesystem
    command: "npx @anthropic/filesystem-server"
```

3. 文件系统代替数据库

记忆存储用 Markdown 文件（MEMORY.md、HISTORY.md），不需要向量数据库或关系型数据库。

4. 标准协议代替自定义抽象

使用 MCP 标准协议，不需要自己定义一套复杂的工具调用抽象层。

面试中如何解释“超轻量级”

“Nanobot 只有约 4000 行 Python 代码，是 LangChain 代码量的 1/125。它之所以能做到这么精简，核心在于三个设计决策：第一，充分利用 LLM 的自主规划能力，不需要额外的任务编排引擎；第二，采用配置驱动的架构，通过 YAML 文件组装功能；第三，使用 MCP 标准协议和文件系统来代替自定义抽象层和数据库。这种设计理念可以类比 Unix 哲学——‘做一件事，把它做好’。”

2.3 核心特性详解

特性一：11+ LLM 供应商支持

Nanobot 通过 Provider 抽象层，统一支持主流 LLM 供应商：

供应商	代表模型	Provider 类型
OpenAI	GPT-4o, GPT-4-turbo	openai
Anthropic	Claude 3.5, Claude 3	anthropic
Google	Gemini 2.0, Gemini 1.5	google
DeepSeek	DeepSeek-V3, DeepSeek-R1	deepseek
通义千问	Qwen-Max, Qwen-Plus	dashscope
Mistral	Mistral-Large	mistral
Groq	Llama 3, Mixtral	groq
Ollama	本地模型	ollama
Azure OpenAI	GPT-4 (Azure)	azure
OpenRouter	多模型路由	openrouter

供应商	代表模型	Provider 类型
自定义	任何兼容 OpenAI 格式	openai_compatible

v0.1-v0.2.1 更新:

实际支持 **39+** **Provider**，主要包括:

国际 **Provider**: OpenAI、Anthropic、Gemini、Mistral、Groq、OpenRouter、HuggingFace 国内 **Provider**: DeepSeek、DashScope (通义千问)、Zhipu (智谱)、Moonshot (Kimi)、MiniMax、StepFun (阶跃星辰)、Skywork 聚合 **Provider**: OpenRouter、AiHubMix、SiliconFlow、Novita 云平台 **Provider**: Azure OpenAI、AWS Bedrock、VolcEngine (火山引擎)、BytePlus **OAuth Provider**: OpenAI Codex、GitHub Copilot 本地 **Provider**: Ollama、vLLM、LM Studio 其他: LongCat、Atomic Chat、NVIDIA NIM 等

配置示例:

```
# 使用 DeepSeek (国产模型, 性价比高)
provider:
  type: deepseek
  model: deepseek-chat
  api_key: ${DEEPSEEK_API_KEY}
```

```
# 使用通义千问
provider:
  type: dashscope
  model: qwen-max
  api_key: ${DASHSCOPE_API_KEY}
```

```
# 使用本地 Ollama 模型 (完全免费)
provider:
  type: ollama
  model: llama3:70b
  base_url: http://localhost:11434
```

v0.1-v0.2.1 更新:

实际使用 JSON 格式配置 (~/.nanobot/config.json):

```
{
  "providers": {"deepseek": {"apiKey": "${DEEPSEEK_API_KEY}"},
  "modelPresets": {"primary": {"provider": "deepseek", "model": "deepseek-chat"}},
  "agents": {"defaults": {"modelPreset": "primary"}}
}
```

面试价值: 展示你对 LLM 生态的广泛了解, 以及对 Provider 模式的理解。

特性二: **8+** 聊天平台支持

这是 Nanobot 的差异化特色——它不只是一个开发框架, 而是一个可以直接部署到各种聊天平台的 Agent 系统。

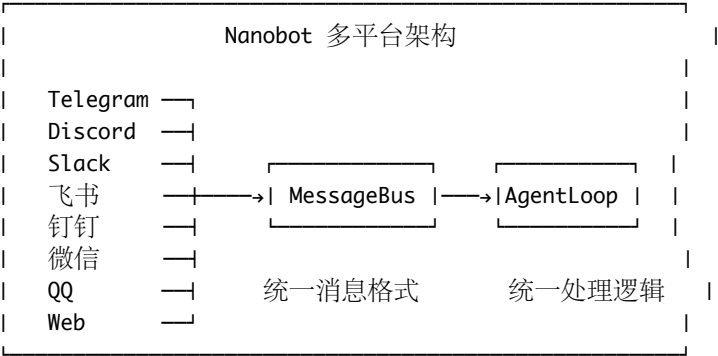
平台	国内/国际	Channel 类型	使用场景
Telegram	国际	telegram	个人/社群
Discord	国际	discord	社区/游戏
Slack	国际	slack	企业协作
飞书	国内	feishu	企业协作
钉钉	国内	dingtalk	企业协作
微信	国内	wechat	个人/企业

平台	国内/国际	Channel 类型	使用场景
QQ	国内	qq	个人/社群
Web	通用	web	通用

v0.1-v0.2.1 更新:

实际支持 15+ Channel:

国际平台: Telegram、Discord、Slack、WhatsApp、Signal、MSTeams、Matrix、Email 国内平台: Feishu (飞书)、DingTalk (钉钉)、Weixin (微信)、Wecom (企业微信)、QQ 其他平台: Napcat、MoChat 通用通道: WebSocket (WebUI)、CLI



设计亮点：所有平台的消息通过 MessageBus 统一为 InboundMessage 格式，AgentLoop 完全不需要关心消息来自哪个平台。这是典型的适配器模式。

特性三：MCP 协议原生支持

Nanobot 从设计之初就内置了 MCP (Model Context Protocol) 支持。

```
# 配置 MCP Server
mcp_servers:
  - name: filesystem
    command: "npx @anthropic/mcp-server-filesystem /home/user"
  - name: github
    command: "npx @anthropic/mcp-server-github"
    env:
      GITHUB_TOKEN: "${GITHUB_TOKEN}"
  - name: database
    url: "http://localhost:3000/mcp"
```

v0.1-v0.2.1 更新:

实际使用 JSON 格式配置:

```
{
  "tools": {
    "mcpServers": {
      "filesystem": {"command": "npx", "args": ["-y", "@anthropic/mcp-server-filesystem", "/home/user"]},
      "github": {"command": "npx", "args": ["-y", "@anthropic/mcp-server-github"], "env": {"GITHUB_PERSONAL_ACCE
    }
  }
}
```

MCP 让 Nanobot 可以接入丰富的工具生态，而不需要为每个工具写自定义代码。详见第 5 章。

特性四：双层记忆系统

Nanobot 的记忆系统简洁而有效：



v0.1-v0.2.1 更新：

记忆文件路径：

```
workspace/
├── memory/
│   ├── MEMORY.md      # 长期记忆
│   ├── history.jsonl   # 会话历史（JSONL格式）
│   ├── .cursor         # 读取游标
│   └── .dream_cursor   # Dream游标
├── SOUL.md             # 全局人格
└── USER.md             # 用户画像
```

新增 Dream 记忆系统（智能摘要）、GitStore 版本控制。

设计亮点： - 透明性：记忆文件就是普通 Markdown，用户可以随时查看和编辑 - 无依赖：不需要向量数据库等外部服务 - **Workspace** 隔离：每个项目有自己的记忆空间 - 虚拟工具：save_memory 不是真正的工具，而是直接在框架内部处理，减少不必要的 I/O

特性五：Skills 技能系统

Nanobot 的技能系统让 Agent 可以动态加载能力：

```
workspace/
└── .nanobot/
```

```
└─ skills/
  └─ code-review/
    └─ SKILL.md      # 代码审查技能
  └─ data-analysis/
    └─ SKILL.md      # 数据分析技能
  └─ writing/
    └─ SKILL.md      # 写作技能
```

每个 SKILL.md 文件定义了：- 技能描述（何时激活）- 详细指令（如何执行）- 约束条件

设计亮点：技能通过 Markdown 文件定义，无需写代码。用户可以创建自定义技能，按需加载到 Agent 的 system prompt 中。

特性六：子 Agent 与定时任务

子 Agent (SubAgent): - 主 Agent 可以使用 SpawnTool 创建后台子 Agent - 子 Agent 有独立的迭代限制（15 次 vs 主 Agent 的 40 次）- 子 Agent 的工具集受限（不能再 Spawn 子 Agent，防止递归）- 适用于后台长时间运行的任务

定时任务 (Cron Jobs):

```
cron:
- schedule: "0 9 * * *"      # 每天早上 9 点
  message: " 总结昨天的工作进展"
- schedule: "0 */6 * * *"    # 每 6 小时
  message: " 检查邮件并通知重要消息"
```

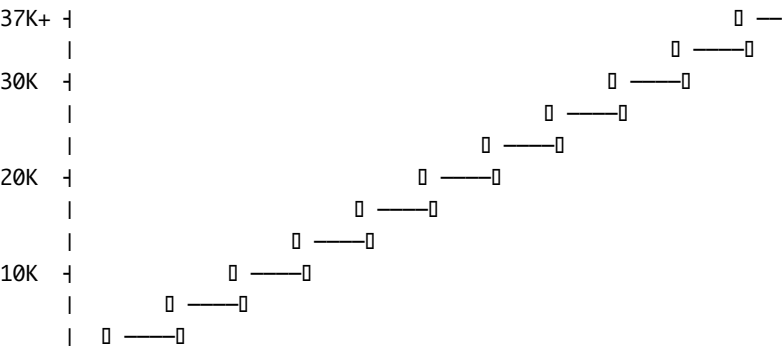
2.4 项目数据与里程碑

关键数据

指标	数据
GitHub Stars	37,000+
首次发布	2026 年 2 月
开源协议	MIT
核心代码量	~4,000 行
编程语言	Python
支持 LLM	11+ 供应商
支持平台	8+ 聊天平台
Contributors	50+
Forks	3,000+

增长轨迹

Stars 增长趋势:



2月初 2月中 2月底 3月初 3月中 3月底 4月
2026

为什么增长这么快？ 1. 时机好：Agent 概念已被市场充分教育，人们需要一个简单实用的框架 2. 定位精准：“超轻量级”这个标签极具吸引力 3. 开箱即用：支持多种国内外平台，降低使用门槛 4. 学术背景：HKUDS 的学术声誉为项目背书 5. 社区驱动：MIT 协议让社区贡献积极

2.5 为什么选 Nanobot 学习

面试价值分析

维度	Nanobot 的优势	其他框架的问题
源码可读性	4000 行，一个周末读完	数万行甚至数十万行，根本读不完
架构理解	五层架构清晰明了	抽象层过多，难以把握全局
设计模式	10+ 经典设计模式可讲	模式混杂，难以提炼
技术热点	MCP 原生支持（面试热门话题）	后来追加，理解不深
项目热度	37K Stars，面试官大概率听过	需要额外解释项目背景
差异化	很少有人深入研究 Nanobot	人人都说学过 LangChain
实战性	可以快速搭建真实的 Agent	搭建过程复杂，Demo 效果一般

“以小见大”的学习策略

通过 Nanobot 这 4000 行代码，你可以理解：

Agent 核心概念

- └ AgentLoop → 理解 Agent 的推理循环
- └ Memory → 理解 Agent 的记忆管理
- └ Tools → 理解 Agent 的工具调用
- └ MCP → 理解工具标准化协议

软件工程思想

- └ 异步编程 → asyncio 实战
- └ 设计模式 → 适配器、注册表、生产者-消费者等
- └ 配置驱动 → YAML 组装系统
- └ 关注点分离 → 层次清晰的模块化设计

系统设计能力

- └ 消息队列 → MessageBus 双队列设计
- └ 并发控制 → 会话锁 + 并发闸门
- └ 插件系统 → Skill/MCP 热加载
- └ 多平台适配 → 适配器模式

与 LangChain 学习路径对比

阶段	学习 Nanobot	学习 LangChain
入门	1 天：跑通示例，理解配置	3 天：理解概念，跑通示例
架构	2 天：通读源码，理解架构	2 周：部分模块源码，理解抽象层
深入	3 天：掌握设计模式和关键实现	1 月：深入部分模块，仍有盲区
面试	1 周内可完成面试准备	需要数周，且难以讲清全局

结论：Nanobot 让你在一周内就能达到面试中“项目深度”的要求，而且因为读过全部源码，面试时任何关于架构设计的追问都能从容回答。

2.6 与其他框架的差异化对比

全维度对比表

维度	Nanobot	LangChain	CrewAI	AutoGPT	OpenClaw
设计哲学	极简、够用	全面、生态	协作导向	全自主	企业全栈
核心代码	~4K 行	~500K 行	~30K 行	~100K 行	~430K 行
学习曲线	1 天上手	1 周入门	3 天入门	3 天入门	1 周入门
Agent 模式	单 Agent+SubAgent	灵活组合	多 Agent 协作	单 Agent 循环	单 Agent/多 Agent
LLM 支持	11+ 供应商	50+ 供应商	依赖 LangChain	主要 OpenAI	多供应商
MCP 支持	原生内置	扩展支持	有限	无	支持
记忆系统	文件系统	多后端	内置	内置	多模态
平台集成	8+	需自行集成	无	Web UI	有限
定时任务	内置 Cron	无	无	无	有限
安全机制	workspace 沙箱	基本	基本	基本	完善
配置方式	YAML	代码	代码 + 装饰器	代码	代码 + 配置
部署难度	简单 (pip)	中等	中等	较复杂	复杂
适合人群	个人/小团队	企业开发者	多 Agent 场景	研究者	企业客户

核心差异解读

Nanobot vs LangChain

LangChain 的设计理念: "提供一切可能需要的工具和抽象"

- └─ 优势: 生态丰富, 几乎什么都能做
- └─ 劣势: 过度封装, 代码复杂, 更新频繁导致 API 不稳定
- └─ 类比: 一个装满工具的大工具箱

Nanobot 的设计理念: "提供恰好够用的核心能力"

- └─ 优势: 代码简洁, 架构清晰, 一目了然
- └─ 劣势: 功能较少, 生态不如 LangChain 丰富
- └─ 类比: 一把精心打造的瑞士军刀

面试说法: > "如果说 LangChain 是一个大型超市, 什么都有但找东西费劲; 那 Nanobot 就是一家精品店, 东西不多但每样都精心挑选。从学习角度, Nanobot 更适理解 Agent 的本质设计, 就像学操作系统不应该从 Linux 内核开始, 而应该从 xv6 这样的教学操作系统开始。"

Nanobot vs AutoGPT

AutoGPT: 追求"完全自主"

- └─ 想法前卫, 但实际可靠性低
- └─ Token 消耗大, 容易陷入循环
- └─ 更像一个概念验证 (PoC)

Nanobot: 追求"实用可靠"

- └─ 明确的迭代限制 (40次/15次) 防止无限循环
- └─ workspace 沙箱保证安全
- └─ 更像一个生产就绪的工具

Nanobot vs CrewAI

CrewAI: 多 Agent 协作框架

- └─ 擅长多个 Agent 角色分工

- └ 但单 Agent 场景下过于复杂
- └ 依赖 LangChain 生态

Nanobot: 以单 Agent 为核心

- └ 通过 SubAgent 实现有限的多 Agent
- └ 单 Agent 场景下简洁高效
- └ 完全独立, 无外部框架依赖

2.7 面试话术

话术一: 介绍你学的 Nanobot 项目

“我深入学习了 HKUDS/nanobot 这个项目, 它是香港大学数据科学实验室开源的超轻量级 AI Agent 框架, GitHub 上有 37K+ Stars。这个项目最大的特点是只用了约 4000 行 Python 代码, 就实现了一个完整的 Agent 框架, 包括消息总线、AgentLoop 推理循环、MCP 协议支持、双层记忆系统、技能加载系统, 以及 8 个以上聊天平台的适配。

我通读了它的全部源码, 重点研究了三个方面: 一是 AgentLoop 的 ReAct 循环实现, 包括会话锁和并发控制; 二是 MCP 协议在框架中的集成方式, 理解了 MCPToolWrapper 如何将远程 MCP 工具包装为本地工具; 三是双层记忆系统的设计, 特别是 save_memory 虚拟工具和 MemoryConsolidator 的压缩机制。”

话术二: 为什么选这个项目而不是 LangChain

“我选择 Nanobot 而不是 LangChain, 主要考虑了三点。第一, 深度胜过广度——Nanobot 只有 4000 行代码, 我可以读完全部源码并理解每一个设计决策, 这在面试中意味着任何追问我都能回答; 第二, Nanobot 的架构更纯粹, 它把 Agent 的核心概念——循环推理、记忆管理、工具调用——用最简洁的方式实现了, 没有 LangChain 那种过度封装的问题; 第三, 它原生支持 MCP 协议, 这是目前 AI 工具调用的标准化方向, 面试中可以展示我对技术趋势的把握。”

话术三: Nanobot 最大的技术亮点是什么

“我认为 Nanobot 最大的技术亮点是它的‘配置驱动 + LLM 自主规划’的设计哲学。很多 Agent 框架花大量代码去实现复杂的任务编排引擎、状态机、DAG 执行器。但 Nanobot 的核心洞察是: LLM 本身就是最好的规划器。所以 Nanobot 只需要实现一个简洁的 ReAct 循环 (AgentRunner), 让 LLM 自己决定调用什么工具、什么顺序、什么时候结束。这种设计让 4000 行代码就能实现其他框架数十万行才能做到的功能。”

2.8 本章总结

核心知识点回顾

本章核心要点	
1. Nanobot 是 HKUDS 开源的超轻量级 Agent 框架	
→ 4000行代码 37K+ Stars MIT 协议	
2. 六大核心特性	
→ 11+ LLM 供应商	
→ 8+ 聊天平台	
→ MCP 协议原生支持	
→ 双层记忆系统	
→ Skills 技能系统	
→ 子Agent + 定时任务	
3. "超轻量"的设计哲学	
→ 利用 LLM 规划能力	

	→ 配置驱动代替代码驱动	
	→ 文件系统代替数据库	
	→ 标准协议代替自定义抽象	
	4. 面试价值	
	→ 源码可读 架构清晰 差异化 热度高	

面试 Checklist

- ☐ 能用 30 秒介绍 Nanobot 项目
- ☐ 能解释“超轻量级”的含义和设计决策
- ☐ 能列举 Nanobot 的 6 大核心特性
- ☐ 能对比 Nanobot 与 LangChain/AutoGPT 的差异
- ☐ 能说出选择 Nanobot 学习的理由

下一章

接下来，我们将深入 Nanobot 的架构设计，理解它的五层架构和四大核心模块。

03 - 架构深入解析

本章小结：Nanobot 是一个“极简但完整”的 AI Agent 框架。它用 4000 行代码证明了：好的架构设计不在于代码多少，而在于是否抓住了问题的本质。对面试者而言，Nanobot 是一个“以小见大”的完美学习素材。